Chair of IT Security
Brandenburg University of Technology

Asya Mitseva, M.Sc.
Prof. Dr.-Ing. Andriy Panchenko

Winter Semester 2025/2026

Part III: Detection Methods & Execution of Experiments **Deadline: 11.12.2025**

Study Project: Adversarial Machine Learning
Use of Reverse Engineering for Anomaly Detection in Industrial Control Systems

Introduction

Industrial Control Systems (ICS) monitor and control critical infrastructures such as chemical plants, power systems, gas pipelines, and water systems. Typically, ICS consist of two macro areas, known as Information Technology (IT) and Operational Technology (OT). While IT is composed of standard computers interconnected via traditional communication protocols, OT includes special hardware and software for monitoring and controlling a given industrial facility. Overall, OT consists of *field*, *control*, and *supervisory* layers. The field layer includes different sensors and actuators, while the control layer is composed of a number of programmable logic controllers (PLCs). The sensors measure the current state of the underlying physical process, which is then reported to the PLCs. The PLCs implement a control logic used to evaluate the obtained sensor readings according to predefined or dynamically adjustable target values. Based on this evaluation, the PLCs may initiate specific commands to one or more actuators in the field layer aiming to adjust the current state of the physical process. In the supervisory layer, a Supervisory Control and Data Acquisition (SCADA) system performs a high-level visualization and control over the control and field layers and provides an overview of the current process state to the operator. While OT was mainly a standalone system in the past, nowadays it incorporates both decades-old devices and communication infrastructure, and new generation embedded devices with computing capabilities and Ethernet-based communication protocols, and is more often connected to the Internet. While the digitization of ICS makes the monitoring and the control of the physical process easier, it creates new attack surfaces against ICS.

A major line of research focused on the design and the development of robust intrusion detection techniques for the identification of cyber attacks in ICS. A particular challenge to be addressed is the usage of non-standardized protocols that are specific to manufactures or even to a particular type of device. Consequently, protocol specifications needed to process network data are not available and, thus, an in-depth analysis of the content of exchanged network traffic is difficult to accomplish.

The goal of this study project is to implement and evaluate different approaches that are able to extract expressive patterns, i.e., *features*, from network traffic collected in the OT area of a given ICS. We assume that the ICS operator utilizes proprietary binary protocols, i.e., the approaches to be developed cannot directly parse the content of transmitted network packets, and the defender relies on reverse engineering methods to derive only specific chunks from the observed network packets. The defender uses different machine learning (ML) methods with these chunks to identify a possible abnormal operation in ICS.

In the previous practical sheet, we implemented your reverse engineering method that will be used to extract only specific chunks of bytes from observed network packets. In the next part of the study project, we implement another detection method and prepare different machine learning algorithms for anomaly detection and compare their performance using the features generated from your autoencoder.

should be done on 03.1

Task 1: N-gram based Detection Method

The goal of this task is to write a piece of code that uses a mixture of higher order n-grams ($n > 1$) to model the bytes of network packets and based on the generated model to detect possible attacks. In particular, during training the method you should observe each distinct n-gram seen in the sequence of bytes in each network packet in the training data and record it in a space efficient Bloom filter. In addition, for each n-gram seen in the training data, you should count and store the number of its appearances in the training data during the training process. Once the training phase terminates, each packet from the testing set should be scored by measuring the number of n-grams that were found in the training phase. In other words, for each packet in the testing set you should compute the following score: $Score = \frac{\sum_i^T w_i \cdot N\_seen_i}{T}$, $Score \in [0, 1]$, where N_seen_i indicates each n-gram in the packet that was seen during training, w_i represents its normalized weight how often this n-gram was seen during training, and T represents the total number of n-grams in the current packet. A network packet should be classified as a malicious if its $Score$ is lower than a predefined threshold. Your implementation should contain the search of an optimal threshold during training the algorithm. For each network packet in the testing set, your piece of code should print out whether this packet contains any attack or not.

Task 2: Preparation of Dataset and Implementation of Traditional Classifiers

In the first practical sheet, we implemented an autoencoder for automatic generation of features from network traffic. The goal of this task is to write a piece of code that implements different machine learning techniques that will be used with the features, generated from your autoencoder, to detect possible attacks in different evaluation scenarios. For this task, you should use the network traffic provided in the QUT_S7Comm dataset and merge the files containing no attacks and the files containing network packets with and without attacks before using them as an input for the items listed below. Moreover, you should satisfy the following requirements:

a) Your piece of code should implement *k-fold cross-validation* in order to divide the initial dataset into training and testing sets. The number of folds k should be a parameter that is read from the command line. In addition, your k-fold cross-validation should cover the following evaluation scenarios:

- In the first scenario, in each iteration of the cross-validation your training data should never contain any attacks and the testing data should always contain both measurements without attacks and measurements representing all attack types (the number of measurements under attack per attack type should be the same).
- In the second scenario, in each iteration of the cross-validation your training data should contain measurements without attacks and measurements representing $n - 2$ different types of attacks, where n indicates the total number of attack types in the dataset. The testing data should still contain all attack types and measurements without attacks (the number of measurements under attack per attack type should be the same in the training and testing sets, respectively).
- In the third scenario, in each iteration of the cross-validation your training data should contain measurements without attacks and measurements representing exactly one type of attack. The testing data should still contain all attack types and measurements without attacks (the number of measurements under attack per attack type should be the same in the training and testing sets, respectively).

Please note that for the implementation of the cross-validation in the different scenarios you are not allowed to use any preexisting library. By using your autoencoder from the first practical sheet, you should generate the corresponding features (i) for the complete network packets (i.e., without applying reverse engineering) and (ii) for the corresponding parts of network packets after applying reverse engineering for each of the evaluation scenarios listed above, i.e., the same k-fold cross-validation should be applied when you generate the features as well. For each of the evaluation scenarios above, you should submit

the generated features in Moodle and it should be clear from the submission which piece of data should be used for training and testing in each iteration of the k-fold cross-validation, where $k = 5$.

b) Write a piece of code that implements the following machine learning techniques:

- One-class and binary Support Vector Machines (SVM)
- Ecliptic envelope and local outlier factor for one-class classification
- Random forests and k-Nearest Neighbors (k-NN) for binary classification

For each evaluation scenario presented above, each of the machine learning algorithms should take as an input a training set to generate a *classification model* and test it against the corresponding testing set. For each set of measurements at a given time step in the testing set, your piece of code should print out whether this set of measurements contains any attack or not.

c) The user should be able to select which machine learning algorithm to use from the command line with which evaluation scenario. When you implement this selection, make sure that you have meaningful combinations of classifiers and evaluation scenarios (i.e., not every classifier listed above can be applied in any of the evaluation scenarios).

Task 3: Execution of Experiments

The goal of this task is to conduct an initial evaluation of your classifiers (implemented above) by using the features created through the autoencoder for each of the evaluation scenarios using cross-validation, defined in Task 1, and by using the corresponding training and testing sets. In particular, you should execute the following experiments:

a) For the first evaluation scenario, you should execute one experiment using features of complete network packets (i.e., without applying reverse engineering) for each of the ML classifiers (with which this execution would be possible) and the corresponding training and testing sets. Make sure that for each classifier you identify and use the optimal grid parameters and compute the *precision* and *recall* for each of the experiments.

b) For the second evaluation scenario, you should execute one experiment using features of complete network packets (i.e., without applying reverse engineering) for each of the ML classifiers (with which this execution would be possible) and the corresponding training and testing set. Make sure that for each classifier you identify and use the optimal grid parameters and compute the *precision* and *recall* for each of the experiments.

c) For the third evaluation scenario, you should execute one experiment using features of complete network packets (i.e., without applying reverse engineering) for each of the ML classifiers (with which this execution would be possible) and the corresponding training and testing set. Make sure that for each classifier you identify and use the optimal grid parameters and compute the *precision* and *recall* for each of the experiments.

d) For the first evaluation scenario, you should execute one experiment using features of parts of network packets for a sequence p of physical readings where $p = 5, 10, 15$ after applying reverse engineering for each of the ML classifiers (with which this execution would be possible), and the corresponding training and testing set. Make sure that for each classifier you identify and use the optimal grid parameters and compute the *precision* and *recall* for each of the experiments.

e) For the second evaluation scenario, you should execute one experiment using features of parts of network packets for a sequence p of physical readings where $p = 5, 10, 15$ after applying reverse engineering for each of the ML classifiers (with which this execution would be possible), and the corresponding training and testing set. Make sure that for each classifier you identify and use the optimal grid parameters and compute the *precision* and *recall* for each of the experiments.

f) For the third evaluation scenario, you should execute one experiment using features of parts of network packets for a sequence p of physical readings where $p = 5, 10, 15$ after applying reverse engineering for each of the ML classifiers (with which this execution would be possible), and the corresponding training and testing set. Make sure that for each classifier you identify and use the optimal grid parameters and compute the *precision* and *recall* for each of the experiments.

Plot all obtained classification results in an appropriate way and submit your plots and the obtained classification results. Describe in details all takeaways that can be concluded after executing the experiments.

Preparation for Q&A Session

Prepare yourself for a Q&A session of 40 minutes. During the Q&A session, you need to give a short overview of libraries, scripts or already existing tools that you have used. Furthermore, you need to make a short demo of your piece of code, explain its operation in detail, and show all plots created. Last but not least, you should be ready to answer spontaneous questions asked by the reviewer.